instituto superior de engenharia de lisboa

LMATE

Programação

Inverno 2026-2027

Trabalho 14

1

Exercício 7 - Muitos descontos

- Considere os dez preços de produtos que se seguem:

```
97 70 15 38 92 30 86 68 6 80
```

- Cada produto está com o seu desconto. Os descontos em percentagem, aplicados a cada produto, pela mesma ordem, são:

```
10 15 5 25 10 10 35 25 10 5
```

- Escreva um programa que calcule o preço a pagar por cada produto.

Solução

- Usar uma lista para os preços e outra para os descontos. E usar um índice, para indexar as duas listas sincronizadamente.

Programa (no ficheiro `exercicio_7_muitos_descontos.py`)

Escreva e execute este programa. Confirme o seu *output*.

```
def calcula_preço_a_pagar(preço, desconto):

    return preço * (1.0 - desconto/100)

# os valores dos descontos são em percentagem

lista_de_preços    = [97, 70, 15, 38, 92, 30, 86, 68, 6,  80]
lista_de_descontos = [10, 15, 5,  25, 10, 10, 35, 25, 10, 5]

for índice in [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]:
    preço         = lista_de_preços[índice]
    desconto      = lista_de_descontos[índice]
    preço_a_pagar = calcula_preço_a_pagar(preço, desconto)
    print(preço_a_pagar)
```

Output

```
87.3
59.5
14.25
28.5
82.8
27.0
55.9
51.0
5.4
76.0
```

Inaceitável

- E se fossem mil produtos? Teria que ser o programador a escrever uma lista com os números de `0` a `999`. E seria inaceitável!

Solução - usar a função *built-in* `range`

- A necessidade de criar índices de listas é tão frequente que existe uma função para o efeito. É a função `range`.

- Quando se passa um único valor à funçao `range`, esse valor tem que ser um número inteiro. E a função `range` retorna a sequência dos índices desde zero até ao valor usado menos um.

Programa (no ficheiro `exercicio_7_muitos_descontos_v2.py`)

Escreva e execute este programa. Confirme o seu *output*.

```
def calcula_preço_a_pagar(preço, desconto):

    return preço * (1.0 - desconto/100)

# os valores dos descontos são em percentagem

lista_de_preços    = [97, 70, 15, 38, 92, 30, 86, 68, 6,  80]
lista_de_descontos = [10, 15, 5,  25, 10, 10, 35, 25, 10, 5]

for índice in range(10):
    print(índice)
    preço         = lista_de_preços[índice]
    desconto      = lista_de_descontos[índice]
    preço_a_pagar = calcula_preço_a_pagar(preço, desconto)
    print(preço_a_pagar)
```

Output

```
0
87.3
1
59.5
2
14.25
3
28.5
4
82.8
5
27.0
6
55.9
7
51.0
8
5.4
9
76.0
```

Inaceitável

- De uma certa forma esta versão ainda é inaceitável. Isto porque se em vez de 10 forem 20 produtos, temos que acrescentar às listas de preços e de descontos os valores adicionais. Isto são os dados do problema. Não há nada a fazer. Têm que ser acrescentados. Mas também, temos que alterar o valor `10` que estamos a passar á função `range`. De `10` para `20`. E isso é que é inaceitável. À exceção dos dados do problema, o programa deveria funcionar para qualquer número de produtos. Sem ter que ser alterado. Imagine-se o que seria, se num supermercado, quando houvesse um produto novo se tivesse que alterar o programa que calcula os valores a pagar pelos produtos. Seria totalmente inaceitável.

Solução - usar a função *built-in* `len`

- A função *built-in* `len` retorna o tamanho de uma sequência, neste caso, o tamanho de uma lista.

Programa (no ficheiro `exercicio_7_muitos_descontos_v3.py`)

Escreva e execute este programa. Confirme o seu *output*.

```
def calcula_preço_a_pagar(preço, desconto):

    return preço * (1.0 - desconto/100)

# os valores dos descontos são em percentagem

lista_de_preços    = [97, 70, 15, 38, 92, 30, 86, 68, 6,  80]
lista_de_descontos = [10, 15, 5,  25, 10, 10, 35, 25, 10, 5]

# daqui para baixo o programa funciona para qualquer número de
# produtos. não tem que ser alterado se o número de produtos variar

print('número de produtos:', len(lista_de_preços))

for índice in range(len(lista_de_preços)):
    print(índice)
    preço         = lista_de_preços[índice]
    desconto      = lista_de_descontos[índice]
    preço_a_pagar = calcula_preço_a_pagar(preço, desconto)
    print(preço_a_pagar)
```

Output

```
número de produtos: 10
0
87.3
1
59.5
2
14.25
3
28.5
4
82.8
5
27.0
6
55.9
7
51.0
8
5.4
9
76.0
```

Para fazer

Considere o programa Python 3 que se segue, assim como o seu *output*. Copie o programa exatamente como está. Os `imports`, bem como as instruções `seed`, `choices` e `randint`, destinam-se exclusivamente à geração de valores pseudoaleatórios, de forma repetível.

```
from random import seed
from random import choices
from random import randint

seed(6126)

r  = [73, 12, 50, 49, 88, 5, 61, 30]
m = [55, 90, 50, 80, 64, 70, 49, 51]
b = u(r, m)
print(r)
print(m)
print(b)

r_1  = choices(range(0, 101), k=randint(10000, 10999))
m_1 = choices(range(0, 101), k=len(r_1))
r_2  = choices(range(0, 101), k=randint(11000, 11999))
m_2 = choices(range(0, 101), k=len(r_2))
r_3  = choices(range(0, 101), k=randint(12000, 12999))
m_3 = choices(range(0, 101), k=len(r_3))
r_4  = choices(range(0, 101), k=randint(13000, 13999))
m_4 = choices(range(0, 101), k=len(r_4))
r_5  = choices(range(0, 101), k=randint(14000, 14999))
m_5 = choices(range(0, 101), k=len(r_5))

print('só para verificação dos valores pseudoaleatórios gerados')
print(r_5[0])
print(r_5[-1])
print(m_5[0])
print(m_5[-1])
```

A função `u` tem dois argumentos. Duas listas de números inteiros, ambas com o mesmo comprimento. A função `u` retorna a quantidade de posições em que os números nas duas listas, na mesma posição, são ambos maiores ou iguais a cinquenta.
O *output* que se segue ilustra o funcionamento descrito.

```
[73, 12, 50, 49, 88, 5, 61, 30]
[55, 90, 50, 80, 64, 70, 49, 51]
3
só para verificação dos valores pseudoaleatórios gerados
40
30
52
100
```

Acrescente ao programa a função `u`, assim como o código que lhe permita indicar se as afirmações que se seguem são verdadeiras ou falsas.

1.1

A quantidade de posições em que os números nas listas `r_3` e `m_3` são ambos maiores ou iguais a cinquenta é `3155`.

1.2

A quantidade de posições em que os números nas listas `r_1` e `m_1` são ambos maiores ou iguais a cinquenta é `2710`.

1.3

A quantidade de posições em que os números nas listas `r_2` e `m_2` são ambos maiores ou iguais a cinquenta é `2766`.

1.4

A quantidade de posições em que os números nas listas `r_5` e `m_5` são ambos maiores ou iguais a cinquenta é `3659`.

1.5

A quantidade de posições em que os números nas listas `r_4` e `m_4` são ambos maiores ou iguais a cinquenta é `3506`.